COMPUTATIONAL BIOLOGY · MODULE 1

Linux System and Environment Management

Foundations for single-cell and sequence analysis workflows — the Linux command line and Conda environment management that every downstream bioinformatics tool depends on.

AUTHOR	Prachya Biswas
PROGRAMME	Computer Science, BRAC University

TOPIC	Linux Fundamentals & Conda Environment Management
SERIES	Computational Biology Meets Cells — Module 1 of 8

Contents

1. What is Linux, and why does bioinformatics run on it?
2. Linux vs Windows — a different mental model
3. Users and Groups
4. Everything in Linux is a File
5. The Directory Tree
6. Absolute vs Relative Paths
7. Naming Conventions & File Types
8. Anatomy of a Linux Command
9. Listing Files & Getting Help
    - ls — list directory contents
    - Help commands
10. Navigating, Copying, Moving
    - pwd — Print Working Directory
    - cd — Change Directory
    - cp — copy
    - mv — move / rename
11. Creating Directories
12. Viewing File Contents
    - cat in practice
    - head / tail
13. Creating Files (touch)
14. Deleting Files & Directories
15. Text Editors — Vim & Nano
    - Vim (vi/vim)
    - Nano
16. File Permissions
    - Octal (numeric) notation
17. Compression & Archiving
    - tar — package/unpackage files into a single archive
    - zip / unzip
18. Wildcard Characters
19. Pipes & Output Redirection
20. Chaining Multiple Commands
21. Task Submission (nohup, qsub)
    - Direct submission — nohup
    - Cluster submission — qsub (SGE job scheduler)
22. Conda: What It Is and Why It Exists
    - Why it was needed
23. Installing Miniconda
24. Conda Channels — the TV analogy
25. Conda Environments
    - Deleting / renaming an environment
26. Common Conda Commands
27. Software Commonly Installed via Conda
28. Hands-on Exercise (with solutions)
29. What's Next

Summary

1. What is Linux, and why does bioinformatics run on it?

Linux is a **free and open-source, UNIX-like operating system**. Its kernel was first released by **Linus Torvalds in 1991**.

- As long as you follow the **GNU General Public License (GPL)**, the entire source code is freely available to any person or institution — you can view it, modify it, and redistribute it.
- It supports **multiple platforms** (servers, laptops, embedded devices, cloud instances).
- It supports **multiple users and multiple tasks at the same time** — several people can be logged into the same machine, each running their own processes, without stepping on each other.

This last point is the real reason bioinformatics runs on Linux almost exclusively: a sequencing core or a research lab shares a handful of powerful servers, and dozens of people need to run large, long jobs (alignment, assembly, variant calling) on them concurrently without interfering with one another. Nearly every bioinformatics tool — BWA, STAR, Trinity, CeleScope, Seurat's dependencies — is built and distributed assuming a Linux/UNIX environment.

---

2. Linux vs Windows — a different mental model

	Windows	Linux
Storage	Drives: Disk C, Disk D, named folders (Pictures, Documents)	Everything starts from one **root directory** `/`
Interaction	Mouse: single/double/right-click, drag-and-drop, Enter	Keyboard: typed commands
How you "ask for help"	—	`source`, `help`, built-in docs; mostly `cd`/`mv`-style commands

The key mental shift: in Windows you *browse* to a location; in Linux you *state* a location (a path) and the shell takes you there directly. Once this clicks, navigating a remote server over SSH — which has no mouse at all — stops feeling unfamiliar.

---

3. Users and Groups

A Linux machine is built from the ground up to be **shared**:

- **Every process** running on the system is run by a particular **user**.
- **Every file** is **owned** by a particular user.
- **Access** to files and directories is **limited by that ownership**.
- Users can be organized into **groups**, so a set of people (e.g., a lab team) can share access to a project's files without opening them to everyone.
- The **superuser (root)** sits above all users and groups, with unrestricted access — used for system administration, not everyday work.

This is why, on a shared bioinformatics server, you'll often see commands fail with "permission denied" — it isn't a bug, it's the system correctly enforcing who owns what.

---

4. Everything in Linux is a File

This is one of the defining design philosophies of UNIX-like systems: **directories, devices, even running processes are all represented as files.** A directory is literally a special file that contains a list of other files. A hard drive, a USB device, your keyboard — all show up somewhere in the filesystem as a file you could, in principle, read from or write to.

Once this clicks, a lot of Linux stops feeling arbitrary — permissions, redirection, and piping all work the way they do *because* everything is a uniform "file" the shell can read from or write to.

5. The Directory Tree

The whole filesystem is one tree, starting at the **root directory** `/`. Standard top-level directories:

```
/
├── bin      → essential command binaries
├── boot     → boot loader files
├── dev      → device files
├── etc      → system configuration files
├── home     → each user's personal directory (e.g. /home/alice)
├── root     → the root user's home directory
├── run      → runtime data
├── sbin     → system (admin) binaries
├── tmp      → temporary files
├── usr      → user-installed programs and libraries
└── var      → variable data (logs, etc.)
```

A real `ls` at the root of a server looks like this:

```
$ ls
bin  data  lib    mnt  run   srv  tmp  usr
boot dev   lib64  opt  sbin  sys  var  home  media  root  proc  lost+found
```

Under `/home`, each user typically gets their own subdirectory (e.g. `/home/alice`, `/home/bob`), which is where that user's personal files and work normally live.

---

6. Absolute vs Relative Paths

Absolute path — starts from `/`, lists every directory you pass through. It is a complete "guidepost" to a file's location and works no matter where you currently are.

```
$ pwd
/mnt/sdd/singleron_lecturer/Windrunner/singleron/software
```

Relative path — does *not* start with `/`. It's interpreted relative to your **current directory**.

Symbol	Meaning
`.`	current directory
`..`	parent (one level up) directory
`~`	the current user's home directory

```
$ cd ..                                          # go up one level
$ cd ./singleron_lecturer/Windrunner/singleron/software   # relative path from here
```

---

7. Naming Conventions & File Types

- All characters are valid **except the forward slash** `/`. If a name contains special characters, wrap it in quotation marks.
- Maximum filename length: **256 characters**.
- Linux is **case-sensitive**: `Singleron` and `singleron` are different files.
- Avoid starting a name with `+`, `-`, or `.`
- Unlike Windows, Linux determines a file's type from its **header information**, not its file extension/suffix — renaming `data.csv` to `data.txt` doesn't actually change what kind of file it is.

A real `ls -l` listing shows each file's type as the first character of its permission string:

```
-rw-rw-r--  1 Windrunner Windrunner  68436 Apr  7 14:41 AA.png
drwxrwxr-x  3 Windrunner Windrunner   4096 Mar 18 09:55 accuracode_project
lrwxrwxrwx  1 Windrunner Windrunner     10 Apr 24 16:39 adduser_2.sh -> adduser.sh
-rwxr--r--  2 Windrunner Windrunner    754 Mar 31 11:29 adduser.sh
```

Symbol	Type
`-`	Regular file
`d`	Directory
`l`	Symbolic link
`b`	Block device file
`c`	Character device file
`p`	Pipe
`s`	Socket

8. Anatomy of a Linux Command

A Linux command is built from three parts, separated by spaces:

```
command  +  options  +  arguments
   ls    -a    /mnt/sdd/singleron_lecturer/Windrunner
```

- **Command** — the program/utility name (e.g. `ls`).
- **Options** — flags that change *how* the command behaves, usually prefixed with `-` or `--`.
- **Arguments** — the targets the command acts on (a file, directory, or pattern). Some commands need an argument; others have sensible defaults and work without one.

```
ls -a   # show hidden files too
ls -l   # detailed (long) listing
ls -d   # show properties of the directory itself, not its contents
```

9. Listing Files & Getting Help

`ls` — list directory contents

```
ls          # multi-column listing
ls -l       # detailed info, one entry per line (permissions, owner, size, date)
ls -a       # include hidden files
```

Help commands

Command	Purpose
`whatis`	one-line description of a command
`which`	shows the location of the executable (and reveals if it's aliased)
`man`	full manual — parameters and usage
`help`	help for the shell's own built-in commands
`info`	similar to `man`, sometimes with more detail

```
$ which ls
alias ls='ls --color=auto'
    /usr/bin/ls

$ man cat
CAT(1)                                  User Commands
NAME
    cat - concatenate files and print on the standard output
```

---

10. Navigating, Copying, Moving

`pwd` — Print Working Directory

Shows the full absolute path of where you currently are.

`cd` — Change Directory

```
cd ~        # go to home directory
cd -        # go to the previous directory
cd ..       # go to parent directory
cd .        # stay put (no-op)
```

`cp` — copy

```
cp file folder             # copy file INTO folder
cp file folder/file2       # copy and rename in one step
cp -r folder1 folder2      # copy an entire directory (recursive — required for directories)
cp -d file1 file2          # copy a symbolic link itself, not the file it points to
```

`mv` — move / rename

```
mv folder1 folder2
# if folder2 already exists  → folder1 moves INSIDE folder2
# if folder2 doesn't exist   → folder1 is renamed to folder2

mv file folder
# same logic: moves into folder if it exists, otherwise renames file → folder
```

---

11. Creating Directories

```
mkdir class_new              # create a single directory

mkdir -p project/1/2/3/4     # -p: create every nested directory needed, in one shot
mkdir -m 777 software        # -m: set permissions at creation time
```

Example — building and verifying a nested structure:

```
$ mkdir -p project/1/2/3/4
$ tree
.
└── project
    └── 1
        └── 2
            └── 3
                └── 4

5 directories, 0 files
```

---

12. Viewing File Contents

Command	Behavior
`cat`	prints the whole file, starting from line 1 — best for **small** files
`tac`	prints the file **backwards** (it's literally `cat` spelled backwards)
`nl`	prints with line numbers
`more`	shows content page by page, **forward only**
`less`	like `more`, but lets you scroll **both forward and backward**
`head`	shows just the first lines
`tail`	shows just the last lines

For a large file, `cat` dumps everything onto the terminal at once and becomes unreadable — that's exactly when `less`/`more` earn their keep, since they load and display the file a page at a time instead of all at once.

Inside `less`/`more`: search with `/keyword`, jump to the next/previous match with lowercase/uppercase `n`, scroll with Page Up/Down or the arrow keys, and press `q` to quit.

`cat` in practice

```
cat -n file                      # print with line numbers
cat file1 file2 file3 > file4    # merge (concatenate) three files into file4
cat file1 >> file2               # append file1's content onto the end of file2
```

Real example — merging two gene-expression tables:

```
$ ls -l
-rw-rw-r-- 1 Windrunner Windrunner 623 Mar 31 14:00 test_diffgenes1.xls
-rw-rw-r-- 1 Windrunner Windrunner 514 Mar 31 14:00 test_diffgenes2.xls

$ cat test_diffgenes1.xls test_diffgenes2.xls > test_diffgenes3.xls

$ ls -l
-rw-rw-r-- 1 Windrunner Windrunner  623 ... test_diffgenes1.xls
-rw-rw-r-- 1 Windrunner Windrunner  514 ... test_diffgenes2.xls
-rw-rw-r-- 1 Windrunner Windrunner 1137 ... test_diffgenes3.xls   # 623 + 514 = 1137 ✓
```

`head` / `tail`

```
head -n 5 test_diffgenes.xls   # first 5 lines
tail -n 5 test_diffgenes.xls   # last 5 lines
head -c 100 file                # first 100 bytes
```

Flag	Meaning (applies to both `head` and `tail`)
`-n <K>`	show K lines instead of the default 10
`-c <K>`	show K bytes instead of lines
`-q`	never print the filename header (useful with multiple files)
`-v`	always print the filename header, even for a single file

```
$ head -n 5 test_diffgenes.xls
gene_id    p_val                  avg_logFC            pct.1  pct.2  p_val_adj
FTH1       2.70952885711447e-11  -0.592499992784135    1      1      5.76560645505388e-07
EIF1       1.57964538445031e-10  -0.527195749736074    0.989  0.993  3.36132741357181e-06
RPL10P9    2.85051172560295e-10   1.47314961280492     0.333  0.03   6.06560390091052e-06
B2M        6.10591489067906e-09  -0.34493234962686     1      1      0.00012992776295876
```

13. Creating Files (touch)

`touch` has **two** distinct uses:

1. Update the **timestamp** of an existing file to the current system time — its content is left untouched.
2. Create a brand-new, **empty** file if none exists by that name.

```
touch test1.txt test2.txt        # create two empty files
touch -c test3.txt                # -c: only update timestamp; never create a new file
touch -t 201808080808 file.txt    # -t: set a specific timestamp (format: YYYYMMDDhhmm)
```

Inspect a file's metadata (size, timestamps, permissions) with `stat`:

```
$ stat test1.txt
  File: test1.txt
  Size: 0         Blocks: 0        IO Block: 4096   regular empty file
Access: 2022-03-30 17:39:34 ...
Modify: 2022-03-30 17:39:34 ...
```

14. Deleting Files & Directories

```
rm -f file          # force delete, no prompt, ignore missing files
rm -r folder         # recursive delete — required for a non-empty directory. VERY DANGEROUS.
rm -i file           # prompt before every deletion (safer)
rmdir emptyFolder    # only deletes an EMPTY directory
```

> **There is no recycle bin in Linux.** A deleted file is gone — permanently. A common safety habit: add `alias rm='rm -i'` to your `.bashrc` so every `rm` always asks for confirmation before it destroys anything.

15. Text Editors — Vim & Nano

Vim (`vi`/`vim`)

The most common text editor on Linux/UNIX systems. It works through **three modes**:

```
vim filename   →  opens in Command Mode
   a / i       →  enter Insert Mode (Esc to leave)
   :           →  enter Last-line Mode
```

1. **Command mode** (default on open) — move the cursor, use shortcuts to copy/paste/delete.
2. **Insert/Edit mode** — reached from Command mode via lowercase `a` or `i`; type freely here; `Esc` returns to Command mode.
3. **Last-line mode** — reached from Command mode via `:`; used to save, search, replace, and quit.

Key/Command	Action
`x`	delete the character under the cursor
`u`	undo the last command; `U` undoes the whole line
`Ctrl+R`	redo
`A`	append text at the end of the line
`dw`	move to the start of a word and delete it
`:wq`	save and quit
`:q!`	discard all changes and quit

Vim is powerful but has a real learning curve — keep a cheat sheet handy: https://vim.rtorr.com/

Nano

GNU nano is a small, friendly, **modeless** editor — unlike Vim, you can start typing and editing immediately after opening a file, with no mode-switching. It supports interactive search-and-replace, go-to-line, auto-indentation, and filename tab completion.

Shortcut	Action
`Ctrl+A` / `Ctrl+E`	jump to start / end of the line
`Ctrl+W`	search for text (`Alt+W` repeats the search)
`Ctrl+K` / `Ctrl+U`	cut the selected line / paste it back
`Ctrl+O`	write out (save)
`Ctrl+X`	exit (prompts to save if there are unsaved changes)
`Ctrl+G`	open the help window

Cheat sheet: https://www.nano-editor.org/dist/latest/cheatsheet.html

---

16. File Permissions

Every file/directory carries permissions for **three** categories of people:

- **owner (u)** — the user who owns the file
- **group (g)** — the group the owner belongs to
- **other (o)** — everyone else

...and **three** kinds of access for each category: **read (r)**, **write (w)**, **execute (x)**.

```
-rw-r--r--
|└┘└┘└┘
| u  g  o
└── file type (- = regular file, d = directory, l = symlink, ...)
```

Octal (numeric) notation

Each permission has a bit value: `r = 4`, `w = 2`, `x = 1`. Add them up per category:

```
        owner(u)    group(g)    other(o)
         rwx          r-x         r--
  bits   421          4_1         4__
  sum     7           5           4
```

So `chmod 755 file` → owner gets read+write+execute, group and others get read+execute only.

```
chmod 755 test.txt          # set permissions numerically
chmod -R 755 test_folder    # -R: apply recursively to everything inside
```

Related commands: - `chown` — change a file's **owner** - `chgrp` — change a file's **group**

`chmod` requires permission to operate on the file in the first place (you generally need to own it, or be root).

---

17. Compression & Archiving

`tar` — package/unpackage files into a single archive

```
tar -cf archive.tar file1 file2    # create (-c) an archive from two files
tar -tvf archive.tar                # list (-t) contents verbosely
tar -xf archive.tar                 # extract (-x)
tar -xf source.tar -C directory     # extract into a specific directory

tar -czf target.tar.gz file1 file2  # create AND gzip-compress in one step
tar -xzf source.tar.gz              # extract a .tar.gz

tar -cjf target.tar.bz2 file1       # create AND bzip2-compress
tar -xjf source.tar.bz2 -C dir      # extract a .tar.bz2
```

> `-f` must be immediately followed by the archive's filename.

`zip` / `unzip`

```
zip -r compressed.zip directory         # compress a directory
unzip -r compressed.zip -d directory    # decompress into a directory
```

---

18. Wildcard Characters

Wildcard	Meaning
`*`	matches any string, including an empty one — 0 or more characters
`?`	matches exactly 1 character
`[abcd]`	matches any single character among a, b, c, d
`[a-z]`	matches any character in the range a–z
`{..}`	a comma-separated, space-free list of alternatives to generate
`[!abcd]`	matches any character **not** in the set

```
ls *.sh*              # every filename containing ".sh" anywhere
ls *.sh* > sh.txt      # save that list into sh.txt
```

---

19. Pipes & Output Redirection

Pipe (`|`) — sends the **standard output (stdout)** of one command as the **standard input (stdin)** of the next.

> Important: a pipe only passes stdout — **not** error output (stderr).

```
command1 | command2
command1 | command2 | command3    # can be chained further
```

Redirection (`>`) — sends a command's output into a file, creating or overwriting it.

```
command > file         # overwrite file with the command's output
command >> file         # append to the file instead of overwriting it
```

---

20. Chaining Multiple Commands

Separator	Behavior
`;`	run every command in sequence regardless of success/failure — the result of one doesn't affect the others
`&&`	run the next command **only if** the previous one succeeded — i.e. keep going only while everything is succeeding
`||`	run the next command **only if** the previous one **failed** — keep trying alternatives until one succeeds

```
cmd1 ; cmd2      # both run, no matter what
cmd1 && cmd2     # cmd2 runs only if cmd1 succeeded
cmd1 || cmd2     # cmd2 runs only if cmd1 failed
```

---

21. Task Submission (nohup, qsub)

The point of this section: **avoid spending too much time tied to the command-line interface** — a long-running bioinformatics job (alignment, assembly) shouldn't die the moment you disconnect or close your terminal.

Direct submission — `nohup`

```
nohup [command to run] &
```

`nohup` = "no hang up" — the process survives after you log out, and `&` puts it in the background immediately.

```
conda activate celescope2
cd /mnt/sdd/singleron_training_class/resources/celescope_test/rna/shell
nohup sh rna_test.sh &
```

Cluster submission — `qsub` (SGE job scheduler)

```
qsub [options] file.sh
```

Option	Meaning
`-V`	pass your current environment variables into the job
`-cwd`	use the current directory as the job's working directory
`-l vf=10g,p=10`	restrict resources: `vf` = memory size (K/M/G), `p` = number of CPUs
`-q`	specify a queue
`-N`	name the task

```
qsub -cwd -V -l vf=10g,p=10 test.sh
```

---

22. Conda: What It Is and Why It Exists

Conda is an open-source **package and environment management system**. It installs multiple versions of software packages — and all of their dependencies — and lets you switch between them cleanly.

- Originally built for **Python**, but equally manages **R, Ruby, Lua, Scala, Java, JavaScript, C/C++, FORTRAN**, and more.
- Works across **Linux, macOS, and Windows**.
- Distributed as **Anaconda** (the full distribution) or **Miniconda** (a minimal installer) — https://www.anaconda.com/

Why it was needed

In the early days of bioinformatics, installing software was genuinely painful:

- `sudo apt` (Ubuntu) or `sudo yum` (CentOS) → fails outright if you're not an administrator — and on a shared server, you almost never are.
- Downloading a **precompiled binary** → often the tool simply doesn't offer one for your platform.
- **Recompiling from source** → you can't read the codebase, can't find the right dependency versions, and the build fails anyway.

Conda solved this by letting any user — no admin rights required — install software with all its dependencies resolved automatically, inside an isolated environment that doesn't touch the rest of the system.

---

23. Installing Miniconda

Official instructions: https://conda.io/projects/conda/en/stable/user-guide/install/index.html

Miniconda installers are published for Windows, macOS (Intel and Apple Silicon), and Linux (x86-64, aarch64, ppc64le, s390x) — each with a published SHA256 hash to verify the download against.

---

24. Conda Channels — the TV analogy

Think of setting up conda like setting up a TV:

Step	TV	Conda
1	Buy a television	Download conda
2	Mount the TV set	Install conda
3	Search for channels on your TV	Configure the conda channel
4	Watch TV	Install and use packages

Channels are where conda looks to find a package — exactly like TV channels decide what content you can watch. The default

channel is like basic cable; adding a channel like `conda-forge` is like subscribing to a service that unlocks a lot more specialized content.

```
conda config --add channels bioconda
conda config --add channels conda-forge
conda config --set show_channel_urls yes
```

Tips: 1. Don't add the same channel repeatedly — redundant channels slow down installs, and **the order you add them in matters** (a channel added later gets **higher** priority, like setting favorites on a remote). 2. Channel configuration is about **where the server (the machine running conda) is located**, not where you physically are. 3. Your channel configuration lives in `~/.condarc`.

```
conda config --get channels     # view in the order they were added
conda config --show channels    # view current priority order
```

Example — after adding channels, `bioconda` (added last) ends up with the **highest** priority:

```
$ conda config --get channels
--add channels 'defaults'     # lowest priority
--add channels 'r'
--add channels 'conda-forge'
--add channels 'bioconda'     # highest priority

$ conda config --show channels
channels:
  - bioconda
  - conda-forge
  - r
  - defaults
```

---

25. Conda Environments

What's an "environment"? An isolated, self-contained set of packages (and often a specific language interpreter version) that doesn't interfere with any other environment on the same machine.

Why create independent environments? - Conda modifies your setup directly — installing everything into one shared space is risky. - **Dependency conflicts are real**: Software A needs Python 2.7, Software B needs Python 3.8 — they cannot coexist in the same environment. - Different project types need different toolchains entirely: DNA-seq, RNA-seq, ChIP-seq, scRNA-seq — each can live in its own clean environment.

Advice: it's best not to install packages directly into the `base` environment unless you fully understand what that package will do to your native setup — it can break your entire conda installation.

```
conda create -n R4             # create a new environment named "R4"
conda activate R4               # enter/activate it
conda deactivate                # exit back to base
conda env list                  # list all environments
conda info --env                # same, with more detail
```

```
$ conda info --env
# conda environments:
#
base          *  /mnt/sdd/Singleron/miniconda3
R4               /mnt/sdd/Singleron/miniconda3/envs/R4
```

Deleting / renaming an environment

```
conda remove -n R4 --all              # delete an environment and everything in it
```

Conda has no direct "rename" command — clone it under the new name, then delete the old one:

```
conda create -n Python2                    # original environment
conda create -n py2 --clone Python2        # clone it as "py2"
conda remove -n Python2 --all              # remove the old one
```

26. Common Conda Commands

```
conda --help                # general help
conda config                # view/edit channel configuration
conda search <package>      # search for a package
conda install <package>     # install a package into the active environment
conda uninstall <package>   # remove a package
conda remove <package>      # same as uninstall
conda update <package>      # update a package to the latest compatible version
conda list                  # list packages in the active environment
```

27. Software Commonly Installed via Conda

Category	Tools
Quality Control (QC)	fastqc, multiqc, fastp, trimmomatic, cutadapt, trim_galore
Alignment & Quantification	bwa, hisat2, bowtie, bowtie2, STAR, salmon, subread (featureCounts)
Genome Assembly	Trinity
Single-cell Transcriptomics	CeleScope

28. Hands-on Exercise (with solutions)

Task 1 — Download the practice data and decompress it:

```
wget https://singleronbio-opendata.oss-cn-hangzhou.aliyuncs.com/Online_training_Materials/linux/linux_traindata.tar.gz
# if the certificate check fails:
wget --no-check-certificate https://singleronbio-opendata.oss-cn-hangzhou.aliyuncs.com/Online_training_Materials/linux/linux_traindata

tar -xzf linux_traindata.tar.gz
```

Task 2 — Create a directory named `linux_test` in your practice path:

```
mkdir linux_test
```

Task 3 — Copy `catfile.txt` and `dog.txt` into `linux_test`, with only ONE line of code:

```
cp catfile.txt dog.txt linux_test/
```

Task 4 — View the first six lines of `dog.txt`:

```
head -n 6 dog.txt
```

Task 5 — Merge `catfile.txt` and `dog.txt` into a single file named `newfile.txt`:

```
cat catfile.txt dog.txt > newfile.txt
```

Task 6 — List files containing the string "file" in their name (inside `linux_test`), and write the output to `test.txt`:

```
ls linux_test/*file* > test.txt
```

Task 7 — Create a folder named `test2`:

```
mkdir test2
```

Task 8 — Copy the `gene` folder into `linux_test`, rename it `GENE`, and compress it into a `.tar.gz`:

```
cp -r gene linux_test/GENE
tar -czf GENE.tar.gz linux_test/GENE
```

Task 9 — Move the compressed `GENE` archive into `test2` and extract it:

```
mv GENE.tar.gz test2/
tar -xzf test2/GENE.tar.gz -C test2/
```

Task 10 — Give `linux_test` read, write, and execute permission for all users:

```
chmod -R 777 linux_test
```

---

29. What's Next

The next session in the track moves into **CeleScope** — Singleron's single-cell transcriptome analysis pipeline. Everything in this module (navigating the filesystem, managing permissions, running long jobs with `nohup`/`qsub`, and creating isolated Conda environments) is a direct prerequisite: CeleScope itself is installed and run inside a dedicated Conda environment, from the Linux command line, on a shared server.

Summary

This module covers the **two foundational skills every bioinformatics pipeline depends on**:

1. **Linux command-line fluency** — navigating, managing, and manipulating files/directories, controlling permissions, and running long jobs reliably on a shared server.
2. **Conda environment management** — installing scientific software without admin rights, and keeping each project's dependencies isolated and reproducible.

Together, these are the prerequisites before moving into actual sequence-analysis tools (FastQC, BWA, STAR, Trinity, CeleScope, Seurat, and beyond).